

get_current_region

REFERENCE

API consumer

read the reader's active regulatory region

Overview

`get_current_region` retrieves the reader's **active regulatory / RF region configuration** in a single round trip: the country and governing regulatory standard, the channel plan (the frequencies the radio may use), the transmit-power limits, and whether Listen-Before-Talk (LBT) and frequency hopping are enabled. It is a read-only query on the device-management (MGMT) endpoint.

The region governs whether — and how — the reader is legally allowed to transmit, so this read is the **compliance gate before any radio operation**. Note that the region can only be **read** over MQTT; it is **set** with a 123RFID application, 123RFID Desktop or 123RFID Mobile, typically at bootstrap. See also The Configuration Model.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_status</code> , <code>get_version</code>

When to Use

Use `get_current_region` when you need to **confirm the reader's regulatory configuration before it transmits** — typically in provisioning verification, a pre-inventory check, or a fleet compliance audit.

- Confirm the region:** verify `country` and `regulatoryStandard` match the deployment location before starting inventory.
- Validate the RF envelope:** check `maxTxPowerSupported` / `minTxPowerSupported`, `channelData`, `lbtEnabled`, and `frequencyHopping` against the constraints your operation expects.

- **Audit compliance across a fleet:** read the region on each device to confirm consistent, compliant settings.
- **After provisioning or a region change:** run `get_current_region` to confirm the new settings took effect before relying on the radio.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `currentRegion.country` / `regulatoryStandard` (response) — the configured country and governing standard (e.g. `FCC`, `ETSI`); confirm they match the deployment location.
- `currentRegion.maxTxPowerSupported` (response) — the power ceiling (dBm); validate transmit-power settings against it before inventory.
- `currentRegion.lbtEnabled` / `frequencyHopping` (response) — compliance behaviors that affect how (and whether) inventory can start.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_current_region" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_current_region	Fixed command name. Retrieves the current region configuration.
2	requestId *	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command` , `requestId` , `apiVersion` , `response` , and a `currentRegion` object. On the **Schema** tab, expand `currentRegion` to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full JSON. `currentRegion` is optional at the envelope level, but on a successful read it is the operative payload and **all seven of its members are required**. `country` and `regulatoryStandard` are free-form (device-driven) — the schema does not constrain them to a fixed vocabulary.

Response payload

```
1      {
2        "command" : "get_current_region" ,
3        "requestId" : "abc123" ,
4        "apiVersion" : "V1.1" ,
5        "currentRegion" : {
6          "frequencyHopping" : true ,
7          "channelData" : [
8            "915750" ,
9            "915250" ,
10           "903250" ,
11           "926750" ,
12           "926250" ,
13           "904250" ,
14           "927250" ,
15           "920250" ,
16           "919250" ,
17           "909250" ,
18           "918750" ,
19           "917750" ,
20           "905250" ,
21           "904750" ,
22           "925250" ,
23           "921750" ,
24           "914750" ,
25           "906750" ,
26           "913750" ,
27           "922250" ,
28           "911250" ,
29           "911750" ,
30           "903750" ,
31           "908750" ,
32           "905750" ,
33           "912250" ,
34           "906250" ,
35           "917250" ,
36           "914250" ,
37           "907250" ,
38           "918250" ,
39           "916250" ,
40           "910250" ,
41           "910750" ,
42           "907750" ,
43           "924750" ,
44           "909750" ,
45           "919750" ,
46           "916750" ,
47           "913250" ,
48           "923750" ,
49           "908250" ,
```

```

50         "925750" ,
51         "912750" ,
52         "924250" ,
53         "921250" ,
54         "920750" ,
55         "922750" ,
56         "902750" ,
57         "923250"
58     ],
59     "country" : "United States" ,
60     "lbtEnabled" : false ,
61     "maxTxPowerSupported" : 30 ,
62     "minTxPowerSupported" : 0 ,
63     "regulatoryStandard" : "FCC"
64 },
65 "response" : {
66     "code" : 0 ,
67     "description" : "Success"
68 }
69 }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_current_region	Echoes the executed command
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command; emitted per-handler, device-wide.
4	currentRegion	object	—	—	The active regulatory region configuration. Optional at the envelope level; when present, a seven members below are required.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	frequency Hopping pinning	boolean	—	true	Whether the radio hops across channel plan (vs a fixed channel). Required in most regions.
6	channel list Data	array	—	["915750", ...]	Channel center frequencies the reader may transmit on, as string in kHz (e.g. 865700 = 865.7 MHz). Count and values are region-specific.
7	country	string	—	USA	The country the device is configured for. Free-form (device driven) — e.g. India, United States.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
8	l b t E n a b l e d	boolean	—	false	Whether Listen-Before-Talk is enabled (required in some ETSI regions; affects inventory start behavior).
9	m a x T x P o w e r S u p p o r t e d	number	—	30	Maximum transmit power supported in the region. Documented in dBm — confirm scale against your firmware if precise power matters.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
10	minimum transmit power supported	number	—	0	Minimum transmit power supported in the region (dBm).
11	regulatory standard	string	—	FCC_USA	The regulatory standard governing transmission rules — e.g. FCC, ETSI, and sometimes a more specific plan identifier such as ETSI_EU_800M_900M_3_CHANNEL. Free-form (device-driven).
12	response	object	—	—	Result wrapper.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
13	c o d e *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
14	d e s c r i p t i o n *	string	—	Success	Human-readable status.

Response Semantics

- `channelData` is a list of channel **center frequencies in kHz**, as strings — e.g. `"865700"` = 865.7 MHz. The count and values depend on the configured region: US / FCC has ~50 channels across 902–928 MHz; ETSI regions typically have fewer, in the 865–868 MHz band. `country` and `regulatoryStandard` are free-form, device-driven strings (no fixed enum).
- `maxTxPowerSupported` / `minTxPowerSupported` are documented in **dBm** (e.g. `30` dBm $\approx$ 1 W). If precise transmit power matters to your integration, confirm the scale against your firmware.

Response Codes

`get_current_region` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — region configuration retrieved and returned.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the region information at this time. Retry after a short delay; if it persists, reboot the device.
101	Default error — generic catch-all.

An **unconfigured region** surfaces as code `6` (`IOT_STATUS_REGION_NOT_CONFIGURED`) at **connect / bootstrap time** — it is not returned by `get_current_region` itself. If a device has no region set, configure it with 123RFID Desktop or 123RFID Mobile before connecting.

Examples

`get_current_region` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The full reply, field by field, is shown under [Response Payload](#) above.

The response shown throughout is the contract's **United States / FCC** example (50 channels across the 902–928 MHz band). The `channelData` list and the `country` / `regulatoryStandard` / `power` values are region-specific — an ETSI region, for example, typically reports fewer channels, in the 865–868 MHz band.

Notes and Caveats

A WRONG REGION CAN PUT THE READER OUTSIDE LOCAL RADIO REGULATIONS

The **region** is set with 123RFID Desktop or 123RFID Mobile, not by any MQTT command — `get_current_region` only reads it. A reader **will not transmit at all until a region is set**, and a reader set to the wrong region will operate but may violate local regulations. If a freshly provisioned reader reports the wrong `country`, redo region setup in 123RFID Desktop or 123RFID Mobile. See [The Configuration Model](#) and [RF Regions and Regulations](#).

- **Read-only over MQTT.** `get_current_region` returns the active region and takes no payload. The region is set with 123RFID Desktop or 123RFID Mobile. This documentation covers 123RFID Desktop, which connects over USB-C before the reader joins a broker.

Run `get_current_region` **after device provisioning or a region change** to confirm the regulatory settings are correct before starting inventory operations.

Related:  [The Configuration Model](#) ·  [get_status](#) ·  [get_version](#) ·  [RF Regions and Regulations](#) ·  [Response Codes](#)